



TD 05 - Gérer la charge avec Nginx

Ce TD a pour objectif de vous familiariser avec NGINX, un composant essentiel de l'écosystème DevOps. À travers des manipulations progressives, vous découvrirez son fonctionnement de base, son rôle de reverse proxy, ainsi que son intégration avec une application personnelle.

Objectifs

À l'issue de ce TD, vous serez capable de :

1. Orchestrer des applications avec Docker Compose.
2. Configurer un serveur web NGINX.

PRÉ-REQUIS

1. Connaissance de base en Spring Boot et des commandes shell.
2. Un environnement de travail prêt avec Git, Java (JDK 17 minimum), un IDE (VS Codium) et Docker.
3. Connaissance de base de Docker ses images et ses conteneurs.

Découverte de NGINX

L'INSTANT WIKIPEDIA

NGINX est un logiciel libre de serveur Web ainsi qu'un proxy inverse écrit par Igor Sysoev, dont le développement a débuté en 2002. C'est depuis 2020, le serveur web le plus utilisé au monde devant Apache .

Un serveur web est un logiciel qui :

- Reçoit des requêtes HTTP/HTTPS.
- Traite ces requêtes et envoie une réponse, par exemple sous forme de page web.
- Peut héberger des fichiers statiques ou des applications dynamiques (PHP, Node.js, Java,...).

Suivez les étapes ci-dessous pour comprendre comment configurer NGINX pour l'utiliser comme un serveur de pages statiques, c'est à dire, comme un serveur web qui se contente d'envoyer des pages web telles quelles sans traitement spécifique.

Commencez par créer un dossier qui va contenir une page html que le serveur doit retourner.

```
mkdir -p nginx-site/html
cd nginx-site
touch html/index.html
```

Ajoutez un contenu à la page `nginx-site/html/index.html`. Par exemple vous pouvez y écrire :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Laboratoires de l'unité 4DOP1DR</title>
</head>
<body>
  <h1>Bienvenue sur mon serveur Nginx avec Docker !</h1>
</body>
</html>
```

Démarrez un conteneur NGINX utilisant les fichiers créés en utilisant la commande suivante dans le dossier `nginx-site`:

```
docker run -d --name nginx-site -p 8080:80 -v
"/${PWD}/html":/usr/share/nginx/html:ro nginx
```

Cette commande se décompose comme suit :

- `docker run` : démarre un nouveau conteneur à partir d'une image.
- `-d` : mode détaché, le conteneur tourne en arrière-plan.
- `--name nginx-site` : donne un nom unique au conteneur.
- `-p 8080:80` : redirige le port 8080 de la machine hôte vers le port 80 du conteneur, port d'écoute par défaut de NGINX.
- `${PWD}` : pour *print working directory* retourne le chemin absolu du répertoire courant.
- `-v "/${PWD}/html":/usr/share/nginx/html:ro` : monte un volume pour lier le dossier local `"/${PWD}/html"` au dossier `/usr/share/nginx/html` du conteneur. Le dossier du

conteneur est en lecture seule (**ro**) ce qui empêche toute modification des fichiers à l'intérieur du conteneur.

- **nginx** : utilise l'image officielle de NGINX depuis Docker Hub.

Si vous allez sur <http://localhost:8080> dans un navigateur, vous devriez voir apparaître le contenu de la page `nginx-site/html/index.html`.

NGINX comme reverse proxy

Vous avez utilisé NGINX comme un serveur web pour servir une page statique. Mais NGINX ne se limite pas à cela. C'est aussi un **reverse proxy**, capable de rediriger les requêtes vers d'autres serveurs ou applications. Dans ce prochain exercice, vous allez transformer votre serveur NGINX en un tel reverse proxy.

Commencez par créer un dossier qui va contenir le fichier de configuration du serveur NGINX.

```
mkdir -p nginx-proxy
cd nginx-proxy
touch nginx.conf
```

Dans le fichier de configuration `nginx-proxy/nginx.conf`, ajoutez :

```
events { }
```

```
http {
    server {
        listen 80;

        location /he2b {
            proxy_pass https://he2b.be/;
        }

        location /documents {
            proxy_pass https://sites.google.com/he2b.be/esl/documents;
        }
    }
}
```

Cette configuration peut être décomposée comme suit :

- `events { }` : Définit des paramètres liés aux connexions réseau, comme le nombre de connexions simultanées. Ce bloc est obligatoire dans un fichier de configuration NGINX,

même si, comme ici, il est vide.

- `http { }` : Configuration principale du serveur HTTP.
- `server { }` : Déclare un serveur HTTP.
- `listen 80;` : Le serveur écoute le port 80, port par défaut pour les requêtes HTTP. Toutes les requêtes reçues sur ce port seront gérées par ce serveur.
- `location /he2b {proxy_pass https://he2b.be/;}` : Quand un utilisateur visite <http://localhost:8081/he2b/>, NGINX redirige la requête vers <https://he2b.be/>, tout en conservant la partie restante de l'URL, par exemple visiter <http://localhost:8081/he2b/etudiant/> redirige vers <https://he2b.be/etudiant/>.

Ce reverse proxy permet d'unifier plusieurs services sous un même domaine. Il peut être amélioré avec l'ajout d'en-têtes HTTP pour mieux gérer la transmission des requêtes ou par l'ajout de la gestion des requêtes HTTPS. N'hésitez pas à consulter [la documentation pour approfondir les possibilités](#).

Démarrez votre reverse proxy via la commande :

```
docker run -d --name nginx-proxy -p 8081:80 -v  
"/${PWD}/nginx.conf":/etc/nginx/nginx.conf:ro nginx
```

Testez que vous avez bien accès aux pages (attention au numéro de port !).

EXERCICE 5 : DÉCRYPTAGE D'UNE COMMANDE

Écrivez sur une feuille la signification de chaque paramètre de la commande afin de vous assurer de votre compréhension de l'utilisation des conteneurs.

NGINX avec une application personnelle

Si vous souhaitez utiliser NGINX pour rediriger vers l'application conteneurisée `demo-no-db`, vous allez devoir demander à Docker de créer un réseau pour que le conteneur NGINX et le conteneur Spring-Boot communiquent.

Si ils existent toujours, commencez par supprimer les conteneurs des serveurs NGINX et de l'application `demo-no-db`.

Ensuite pour créer le réseau il suffit d'utiliser la commande :

```
docker network create my-network-test
```

L'étape suivante consiste à démarrer un conteneur utilisant l'image de l'application `g12345/spring-demo-no-db` en la connectant à ce réseau grâce à l'option `--network` :

```
docker run --rm --name no-db --network my-network-test -p 8082:8080
g12345/spring-demo-no-db
```

Modifiez la configuration du reverse proxy pour que la route `demo-no-db` redirige vers la route `/config` de l'application `demo-no-db`.

nginx.conf

```
events { }

http {
    server {
        listen 80;

        location /he2b {
            proxy_pass https://he2b.be/;
        }

        location /demo-no-db {
            proxy_pass http://no-db:8080/config;
        }
    }
}
```

Finalement démarrer un conteneur NGINX associé au réseau. Prenez attention que cette commande s'exécute dans le dossier contenant le fichier `nginx.conf` vu la présence du `${PWD}`.

```
docker run -d --name nginx-proxy --network my-network-test -p 8081:80 -v
"/${PWD}/nginx.conf":/etc/nginx/nginx.conf:ro nginx
```

Si vous consultez dans un browser l'url `http://localhost:8081/demo-no-db` vous devriez recevoir les données du service rest.

❗ DEUX CHEMINS VERS LE MÊME SERVICE

Le service rest peut être atteint via deux url différentes :

1. `http://localhost:8081/demo-no-db`

2. `http://localhost:8082/config`

Pouvez-vous expliquer ce qui permet à ces deux chemins d'exister ?

❗ LE BON PORT

Lorsque nous avons lancé le conteneur contenant l'application Spring, nous avons utilisé l'option `-p 8082:8080`. Comprenez-vous pourquoi nous avons utilisé le port `8080` dans le fichier `nginx.conf` ?

Supprimez les conteneurs créés avant de passer à l'étape suivante.

Simplification grâce à Docker Compose

Docker vous a permis de démarrer un conteneur pour votre application et un conteneur pour votre reverse proxy NGINX. Docker compose permet quant à lui de réaliser le démarrage de ces deux applications en **un seul fichier** `docker-compose.yml`.

docker-compose.yml

```
services:
  app:
    # Chemin vers le répertoire contenant le Dockerfile
    build: ./
    image: g12345/spring-demo-no-db
    container_name: no-db
    ports:
      - "8082:8080"
    networks:
      - app-network

  nginx:
    image: nginx:latest
    container_name: nginx-proxy
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "8080:80"
    depends_on:
      - app
    networks:
      - app-network
```

```
networks:  
  app-network:  
    driver: bridge
```

Adaptez le nom des images ou des dossiers du fichier `docker-compose.yml` et essayez d'utiliser ce fichier avec la commande `docker-compose up`. Vous devriez pouvoir consommer le service rest de l'application `demo-no-db` à l'adresse `http://localhost:8080/demo-no-db`.

Consultez ensuite les conteneurs disponibles via `docker ps -a`. Vous devriez y trouver les conteneurs concernant NGINX et demo-no-db. Utilisez la commande `docker-compose down` et consultez à nouveau la liste des conteneurs disponibles. Que constatez-vous ?



FORCER LA RECONSTRUCTION

La commande `docker-compose up` démarre les conteneurs définis dans le fichier `docker-compose.yml`. Si les images nécessaires n'existent pas, elles sont construites sinon elles ne sont pas reconstruites et la version disponible sur l'hôte est utilisée. Si vous ajoutez l'option `--build` la **reconstruction** des images est **forcée** même si elles existent déjà.

```
docker-compose up --build
```

NGINX comme load balancer

Dans l'exercice précédent, vous avez configuré NGINX comme reverse proxy pour une application Spring Boot. Cependant, lorsqu'une application reçoit un grand nombre de requêtes, un seul serveur Spring Boot peut devenir un goulet d'étranglement et ralentir les performances. Pour y remédier, il est courant d'utiliser plusieurs instances de l'application et de répartir la charge entre elles.

Dans cet exercice, vous allez utiliser Docker Compose pour configurer NGINX en tant que Load Balancer. L'objectif est de :

- Lancer plusieurs instances de l'application Spring Boot dans des conteneurs Docker.
- Configurer NGINX pour équilibrer la charge entre ces instances.
- Vérifier que les requêtes sont distribuées entre les différentes instances.

Le fichier `docker-compose.yml` pour cette configuration est disponible ci-dessous.

```
docker-compose.yml
```

```
services:
  app1:
    build: ./
    container_name: demo-no-db-1
    environment:
      - SERVER_PORT=8081
    networks:
      - app-network

  app2:
    build: ./
    container_name: demo-no-db-2
    environment:
      - SERVER_PORT=8082
    networks:
      - app-network

  app3:
    build: ./
    container_name: demo-no-db-3
    environment:
      - SERVER_PORT=8083
    networks:
      - app-network

  nginx:
    image: nginx:latest
    container_name: nginx-load-balancer
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "8080:80"
    depends_on:
      - app1
      - app2
      - app3
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

La configuration du serveur NGINX pour l'utiliser comme load balancer est la suivante :


```
events { }

http {
    upstream spring-backend {
        server app1:8081;
        server app2:8082;
        server app3:8083;
    }

    server {
        listen 80;

        location /demo-no-db {
            proxy_pass http://spring-backend/config;
        }
    }
}
```

Démarrez vos conteneurs et lancez plusieurs fois la commande `curl` `http://localhost:8080/demo-no-db` pour consommer le service rest de l'application `demo-no-db`.

Est-ce que le résultat affiché est toujours le même ?